

[Inserir logo do CNPEM / Ilum aqui]

Cluster Heisenberg

Guia do Usuário

Do Terminal ao Primeiro Job

Endereço: `heisenberg.cnpem.br`
Chamados: <https://web-ilum.cnpem.br/chamado>
Reset de senha: `http://172.20.60.13`

Sumário

1	Introdução ao Cluster Heisenberg	3
1.1	Arquitetura Geral	3
2	Infraestrutura de Hardware	3
2.1	Nós de Trabalho	3
3	Acesso ao Cluster	4
3.1	Pré-requisitos	4
3.2	Conectando via SSH	4
3.2.1	Linux e macOS	4
3.2.2	Windows	4
3.3	Reset de Senha	5
3.4	Abertura de Chamados	5
4	Tutorial de Linux para Iniciantes	5
4.1	O que é o Terminal?	5
4.2	Navegando pelo Sistema de Arquivos	6
4.2.1	Saber onde você está	6
4.2.2	Listar arquivos	6
4.2.3	Entrar em um diretório	6
4.2.4	Criar diretórios	7
4.3	Manipulação de Arquivos	7
4.3.1	Criar um arquivo de texto	7
4.3.2	Visualizar conteúdo de arquivos	7
4.3.3	Copiar, mover e remover	7
4.4	Transferência de Arquivos	8
4.4.1	Enviar arquivos para o cluster	8
4.4.2	Baixar arquivos do cluster	8
4.4.3	Transferência com Interface Gráfica: WinSCP (Windows)	8
4.5	Processos e Monitoramento	9
4.6	Busca e Filtros	9
4.7	Redirecionamento e Pipes	9
4.8	Dicas de Produtividade no Terminal	9
4.9	Executar Programas em Segundo Plano	10
5	Environment Modules	10
5.1	Comandos Básicos de Módulos	10
5.2	Módulos Disponíveis no Heisenberg	10
5.2.1	Aplicações Científicas (apps)	11
5.2.2	Bibliotecas e Compiladores (libraries)	11
5.3	Exemplo: Carregando o GROMACS	11
6	Python com Miniconda	12
6.1	Instalando o Miniconda	12
6.2	Criando o Ambiente ilumpy	12
6.3	Instalando Bibliotecas Científicas via pip	13
6.4	Ativando o Ambiente Automaticamente	13
6.5	Comandos Úteis do Conda	13

7	Gerenciador de Filas Slurm	14
7.1	Filas (Partitions) Disponíveis	14
7.2	Comandos Essenciais do Slurm	14
7.2.1	Verificar estado das filas	14
7.2.2	Ver jobs em execução	15
7.3	Escrevendo um Script de Job	15
7.3.1	Parâmetros Principais do #SBATCH	15
7.4	Exemplos de Scripts de Job	15
7.4.1	Job Simples (CPU, serial)	15
7.4.2	Job com MPI (múltiplos processos)	16
7.4.3	Job com GPU	16
7.4.4	Job GROMACS (exemplo completo)	17
7.4.5	Job VASP	17
7.5	Jobs Python	17
7.5.1	Job Python Serial	18
7.5.2	Job Python Paralelo	18
7.6	Submetendo e Acompanhando o Job	19
7.7	Sessão Interativa	19
7.8	Variáveis de Ambiente do Slurm	20
8	Boas Práticas	20
8.1	Organização de Arquivos	20
8.2	Estimativa de Recursos	20
8.3	Cotas e Limites	21
8.4	Etiqueta no Uso do Cluster	21
9	Guia Rápido de Referência	21
9.1	Cheatsheet de Linux	21
9.2	Cheatsheet do Slurm	21
9.3	Cheatsheet de Módulos	22
9.4	Cheatsheet do Conda / Python	22
10	Suporte e Contato	22
A	Glossário	23
B	Checklist do Primeiro Uso	24

1 Introdução ao Cluster Heisenberg

O **Cluster Heisenberg** é a infraestrutura de computação de alto desempenho (*High-Performance Computing* — HPC) disponibilizada pelo CNPEM para pesquisadores e estudantes da Ilum. Com ele, você pode executar simulações, análises de dados e treinamentos de modelos que seriam inviáveis em um computador pessoal, pois o cluster distribui o trabalho entre dezenas de processadores e placas de vídeo (GPUs) de forma coordenada.

○ Informação

O cluster utiliza o gerenciador de filas **Slurm** (*Simple Linux Utility for Resource Management*) e o sistema de módulos **Environment Modules** para carregar softwares científicos. Ambos serão explicados em detalhe neste guia.

1.1 Arquitetura Geral

[Inserir diagrama: Usuário → Nó de Login → Nós de Trabalho
work0–work8]

Figura 1: Arquitetura simplificada do Cluster Heisenberg.

O cluster é composto por:

- **Nó de login (headnode):** `heisenberg.cnpem.br` — o único ponto de acesso externo. É aqui que você faz login, edita scripts e submete jobs. **Não execute simulações pesadas diretamente nele.**
- **Nós de trabalho (worker nodes):** `work0` a `work8` — onde os jobs de fato rodam, gerenciados automaticamente pelo Slurm.

2 Infraestrutura de Hardware

2.1 Nós de Trabalho

A tabela abaixo resume o hardware de cada nó disponível no cluster.

Nó	CPUs	RAM (GB)	Sockets	GPU	Filas
work0	128	503	2 × 64 cores	—	cpu
work1	128	503	2 × 64 cores	3 × NVIDIA T4	cpu, gpu
work2	64	251	2 × 32 cores	—	cpu
work3	64	251	2 × 32 cores	—	cpu
work4	16	125	1 × 16 cores	1 × NVIDIA RTX 4090	gpu

Nó	CPUs	RAM (GB)	Sockets	GPU	Filas
work5	16	125	1 × 8 cores	2 × NVIDIA RTX 4090	gpu
work6	16	125	1 × 8 cores	2 × NVIDIA RTX 4090	gpu
work7	32	188	1 × 16 cores	—	cpu
work8	48	157	2 × 12 cores	1 × NVIDIA RTX 3060	gpu

Tabela 1: Hardware dos nós de trabalho do Cluster Heisenberg.

○ Informação

O nó `work7` encontra-se ocasionalmente em manutenção (*down*). Consulte o estado atual com `sinfo` antes de planejar seus jobs.

3 Acesso ao Cluster

3.1 Pré-requisitos

Para acessar o Heisenberg você precisará de:

1. Uma conta ativa no sistema da Ilum/CNPEM;
2. Um cliente SSH instalado no seu computador;
3. Estar conectado à rede da Ilum (presencialmente ou via VPN).

3.2 Conectando via SSH

SSH (*Secure Shell*) é o protocolo padrão para acessar servidores remotos com segurança pelo terminal.

3.2.1 Linux e macOS

Abra o terminal e execute:

```
$ ssh seu_usuario@heisenberg.cnpem.br
```

Substitua `seu_usuario` pelo seu login institucional.

3.2.2 Windows

No Windows 10/11, o SSH já vem instalado. Abra o **PowerShell** ou o **Terminal do Windows** (tecle Win + R, digite `powershell` e pressione Enter) e use o mesmo comando:

```
PS C:\Users\voce> ssh seu_usuario@heisenberg.cnpem.br
```

[Inserir screenshot: PowerShell executando o comando ssh e exibindo o prompt do cluster após o login]

Figura 2: Conexão SSH pelo PowerShell no Windows.

3.3 Reset de Senha

Caso tenha esquecido sua senha, acesse o portal de reset pelo navegador:

<http://172.20.60.13>

△ Atenção

O portal de reset de senha só é acessível dentro da rede da Ilum ou via VPN.

3.4 Abertura de Chamados

Para reportar problemas técnicos, solicitar novos softwares ou tirar dúvidas com a equipe de TI, utilize o sistema de chamados:

<https://web-ilum.cnpem.br/chamado>

4 Tutorial de Linux para Iniciantes

○ Informação

Esta seção é voltada para quem nunca usou um terminal antes. Se você já tem experiência com Linux, pode pular para a Seção 5.

4.1 O que é o Terminal?

O terminal (também chamado de *shell* ou *linha de comando*) é uma interface de texto onde você digita comandos para o computador executar. No cluster, **toda interação é feita pelo terminal** — não há área de trabalho gráfica.

[Inserir screenshot: terminal após login, mostrando o prompt
“usuario@heisenberg:~\$”]

Figura 3: Prompt do terminal após o login no Heisenberg.

Após o login, você verá algo como:

```
usuario@heisenberg:~$
```

Isso significa: `usuario` = seu nome de usuário; `heisenberg` = nome da máquina; `~` = você está na sua pasta pessoal (*home*); `$` = pronto para receber comandos.

4.2 Navegando pelo Sistema de Arquivos

O Linux organiza arquivos em uma árvore de diretórios. A raiz é `/`. Sua pasta pessoal fica em `/home/seu_usuario` e pode ser referenciada pelo atalho `~`.

4.2.1 Saber onde você está

```
$ pwd
/home/joao
```

`pwd` (*print working directory*) mostra o diretório atual.

4.2.2 Listar arquivos

```
$ ls
dados/  scripts/  simulacao.sh

$ ls -lh
total 12K
drwxr-xr-x 2 joao joao 4096 jun  1 10:00 dados
drwxr-xr-x 2 joao joao 4096 jun  1 10:01 scripts
-rw-r--r-- 1 joao joao  512 jun  3 14:22 simulacao.sh
```

`-l` exibe detalhes (permissões, tamanho, data); `-h` exibe tamanhos legíveis (KB, MB).

4.2.3 Entrar em um diretório

```
$ cd dados
$ pwd
/home/joao/dados

$ cd ..      # volta um nivel acima
$ cd ~      # volta para a home
$ cd -      # volta para o diretorio anterior
```

4.2.4 Criar diretórios

```
$ mkdir meu_projeto
$ mkdir -p meu_projeto/resultados/graficos # cria toda a arvore
```

4.3 Manipulação de Arquivos

4.3.1 Criar um arquivo de texto

```
$ nano meu_arquivo.txt
```

O [nano](#) é um editor de texto simples que funciona direto no terminal.

[Inserir screenshot: editor nano com arquivo aberto e barra de atalhos na parte inferior]

Figura 4: Editor de texto nano no terminal.

Atalhos do [nano](#):

- Ctrl+O
rightarrow salvar (*O de Output*)
- Ctrl+X
rightarrow sair
- Ctrl+W
rightarrow buscar texto

4.3.2 Visualizar conteúdo de arquivos

```
$ cat arquivo.txt # exibe o arquivo inteiro
$ less arquivo.txt # paginado (q para sair, /termo para buscar)
$ head -20 arquivo.txt # primeiras 20 linhas
$ tail -20 arquivo.txt # ultimas 20 linhas
$ tail -f log.out # acompanha o arquivo em tempo real
```

4.3.3 Copiar, mover e remover

```
$ cp arquivo.txt copia.txt # copiar arquivo
$ cp -r pasta/ pasta_backup/ # copiar pasta inteira
$ mv arquivo.txt novo_nome.txt # renomear ou mover
$ mv arquivo.txt /home/joao/dados/ # mover para outra pasta

$ rm arquivo.txt # remover arquivo
```



```
$ rm -r pasta/ # remover pasta e tudo dentro
```

× Cuidado

O comando **rm** apaga arquivos **permanentemente** no Linux — não existe lixeira no terminal. Use com muito cuidado, especialmente com **-r** (recursivo). Nunca execute **rm -rf /** ou **rm -rf *** sem ter certeza absoluta do que está fazendo.

4.4 Transferência de Arquivos

Os comandos abaixo são digitados no **terminal do seu próprio computador** (PowerShell no Windows ou terminal no Linux/macOS), **não** no cluster. Substitua **james** pelo seu usuário.

4.4.1 Enviar arquivos para o cluster

```
# Enviar um arquivo para a home do cluster
$ scp foo.bar james@heisenberg.cnpem.br:~/

# Enviar um arquivo para uma subpasta especifica
$ scp foo.bar james@heisenberg.cnpem.br:~/dados/

# Enviar uma pasta inteira (-r de recursivo)
$ scp -r minha_pasta/ james@heisenberg.cnpem.br:~/
```

4.4.2 Baixar arquivos do cluster

```
# Baixar um arquivo para o diretorio atual
$ scp james@heisenberg.cnpem.br:~/foo.bar ./

# Baixar um arquivo de uma subpasta
$ scp james@heisenberg.cnpem.br:~/resultados/output.dat ./

# Baixar uma pasta inteira
$ scp -r james@heisenberg.cnpem.br:~/resultados/ ./
```

★ Dica

No Windows, esses comandos funcionam exatamente da mesma forma dentro do PowerShell. Não é necessário nenhum programa adicional.

4.4.3 Transferência com Interface Gráfica: WinSCP (Windows)

Para quem prefere uma interface gráfica de arrastar e soltar, o **WinSCP** é uma alternativa prática no Windows. Ele permite navegar pelas pastas do cluster e transferir arquivos sem usar o terminal.

[Inserir screenshot: WinSCP com conexão ao Heisenberg aberta, mostrando arquivos locais à esquerda e arquivos do cluster à direita]

Figura 5: WinSCP: transferência de arquivos com interface gráfica.

Para um tutorial completo de instalação e uso do WinSCP com o Heisenberg, assista ao vídeo:

https://www.youtube.com/watch?v=2_rJNksIYjk

4.5 Processos e Monitoramento

```
$ top          # monitor de processos em tempo real (q para sair)
$ htop         # versao melhorada do top (se disponivel)
$ ps aux | grep meu_usuario # listar seus processos
$ kill 12345    # encerrar processo com PID 12345
```

4.6 Busca e Filtros

```
$ grep "erro" log.out          # busca a palavra "erro" no arquivo
$ grep -r "funcao" scripts/    # busca recursiva em uma pasta
$ find . -name "*.py"          # encontra arquivos .py a partir do diretorio
    atual
$ wc -l arquivo.txt            # conta linhas de um arquivo
```

4.7 Redirecionamento e Pipes

```
$ ls -lh > lista.txt           # redireciona saida para arquivo (sobrescreve)
$ ls -lh >> lista.txt          # redireciona e acrescenta ao arquivo
$ cat log.out | grep "ERRO"    # filtra linhas com "ERRO" do log
$ sort numeros.txt | uniq      # ordena e remove duplicatas
```

4.8 Dicas de Produtividade no Terminal

★ Dica

- **Tab** — completa automaticamente nomes de arquivos e comandos.
- **Seta** ↑/↓ — navega pelo histórico de comandos.
- **PageUp** / **PageDown** — busca no histórico pelo que já foi digitado. Por exemplo, se você digitar **srun** e pressionar **PageUp**, o terminal mostrará o último comando que começava com **srun**.
- **Ctrl+C** — cancela o comando em execução.
- **Ctrl+L** — limpa a tela.
- **Ctrl+A** / **Ctrl+E** — vai para o início/fim da linha.
- **history** — exibe os últimos comandos digitados.

- `man ls` — exibe o manual de qualquer comando (`q` para sair).

4.9 Executar Programas em Segundo Plano

Normalmente, quando você executa um comando no terminal, ele ocupa o prompt até terminar. Para liberar o terminal enquanto o programa ainda roda, use o `&` ao final do comando:

```
$ programa &
[1] 18423
```

O número entre colchetes é o número do job e o seguinte é o PID do processo. O programa continua rodando em segundo plano enquanto você usa o terminal normalmente.

O problema do `&` sozinho é que, se você fechar a sessão SSH, o programa é encerrado junto. Para evitar isso, use `nohup` (*no hang up*), que desconecta o processo da sessão:

```
$ nohup programa > saida.out 2> erros.err &
[1] 18424
```

O `> saida.out` redireciona a saída padrão e `2> erros.err` redireciona os erros, pois com `nohup` o terminal não está mais disponível para exibir nada. O programa continuará rodando mesmo após você fechar o SSH.

Para gerenciar processos em segundo plano:

```
$ jobs          # lista os processos em background desta sessao
$ fg 1          # traz o job [1] de volta para o foreground
$ kill 18424    # encerra o processo pelo PID
```

△ Atenção

O `nohup` serve para tarefas leves e rápidas, como scripts de pré ou pós-processamento. **Não rode simulações pesadas diretamente no nó de login**, independentemente de usar `nohup` ou não. Para isso, utilize sempre o Slurm.

5 Environment Modules

O sistema de **Environment Modules** permite carregar e descarregar softwares científicos de forma isolada, sem conflitos entre versões. Quando você carrega um módulo, o sistema ajusta automaticamente variáveis como `PATH`, `LD_LIBRARY_PATH` e outras necessárias para o software funcionar.

5.1 Comandos Básicos de Módulos

```
$ module avail          # lista todos os modulos disponiveis
$ module avail gromacs  # filtra por nome
$ module load gromacs/2026.0 # carrega o modulo
$ module list           # mostra modulos carregados
$ module unload gromacs/2026.0 # descarrega o modulo
$ module purge          # descarrega todos os modulos
$ module show gromacs/2026.0 # exibe o que o modulo configura
$ module help           # ajuda
```

5.2 Módulos Disponíveis no Heisenberg

5.2.1 Aplicações Científicas (apps)

Módulo	Versão	Descrição
amber	2024	Simulação de dinâmica molecular (biomoléculas)
gromacs	2026.0	Dinâmica molecular de alta performance
ORCA	6.1.1	Química quântica ab initio / DFT
quantum-espresso	7.5	DFT para sólidos e superfícies
quantum-espresso	7.5-aocl	Idem, otimizado com AOCL
vasp	6.2.0	Simulação ab initio de materiais
vasp	6.2.0-aocl	Idem, otimizado com AOCL

Tabela 2: Aplicações científicas disponíveis.

5.2.2 Bibliotecas e Compiladores (libraries)

Módulo	Versão	Descrição
cuda	12.4, 13.1	NVIDIA CUDA Toolkit (programação GPU)
aocl	5.3.0 / 5.3.0-mt / 5.3.0-st	AMD Optimizing CPU Libraries
elpa	2024.05.001	Eigenvalue SoLvers para HPC
hwloc	2.12.2	Topologia de hardware
intel/compiler	2025.3.0	Intel oneAPI C/C++/Fortran
intel/compiler-intel-llvm	2025.3.0	Intel LLVM compiler
intel/mkl	2025.3	Intel Math Kernel Library
intel/mpi	2021.17	Intel MPI
intel/tbb	2022.3	Intel Threading Building Blocks
intel/dnnl	3.9.1	Deep Neural Network Library
intel/debugger	2025.3.0	Intel Debugger
openmpi	4.1.8, 5.0.9	Open MPI (comunicação entre processos)
pmix	5.0.9	Process Management Interface
scalapack	2.2.0-ompi509	ScaLAPACK (álgebra linear paralela)

Tabela 3: Bibliotecas e compiladores disponíveis.

5.3 Exemplo: Carregando o GROMACS

```
$ module load gromacs/2026.0
$ gmx --version
```

★ Dica

Para garantir que o ambiente correto seja carregado toda vez que você fizer login, adicione o comando `module load` ao arquivo `~/.bashrc`:

```
$ nano ~/.bashrc
# Adicione ao final:
module load gromacs/2026.0
```

6 Python com Miniconda

Miniconda é um instalador minimalista do gerenciador de pacotes **conda**. Ele permite criar **ambientes virtuais** isolados, onde cada projeto pode ter sua própria versão do Python e suas próprias bibliotecas, sem interferir no sistema ou em outros projetos.

○ Informação

O Miniconda é instalado **na sua área pessoal** (~/), sem necessidade de permissão de administrador. Cada usuário instala e gerencia seu próprio ambiente.

6.1 Instalando o Miniconda

Passo 1 — Baixar o instalador:

```
$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

Passo 2 — Executar o instalador:

```
$ bash Miniconda3-latest-Linux-x86_64.sh
```

O instalador fará algumas perguntas:

- **Do you accept the license terms?**
rightarrow Digite **yes** e pressione **Enter**.
- **Confirm install location**
rightarrow Pressione **Enter** para aceitar o padrão (~/.miniconda3).
- **Do you wish the installer to initialize Miniconda3?**
rightarrow Digite **yes** e pressione **Enter**. Isso configura o conda automaticamente no seu shell.

Passo 3 — Recarregar o shell:

```
$ source ~/.bashrc
```

Após isso, o prompt mostrará (base) antes do seu nome de usuário, indicando que o conda está ativo.

Passo 4 — Verificar a instalação:

```
$ conda --version
conda 24.x.x
```

6.2 Criando o Ambiente ilumpy

Com o conda instalado, crie um ambiente dedicado com Python 3.11:

```
$ conda create -n ilumpy python=3.11
```

O conda listará os pacotes que serão instalados e pedirá confirmação. Digite **y** e pressione **Enter**.
Ative o ambiente:

```
$ conda activate ilumpy
```

O prompt mudará para (ilumpy), confirmando que o ambiente está ativo:

```
(ilumpy) usuario@heisenberg:~$
```

Verifique a versão do Python:

```
$ python --version
Python 3.11.x
```

6.3 Instalando Bibliotecas Científicas via pip

Com o ambiente `ilumpy` ativo, instale as bibliotecas mais utilizadas em Python científico com um único comando:

```
$ pip install numpy scipy matplotlib pandas seaborn scikit-learn \
    jupyter ipython sympy tqdm h5py pyarrow statsmodels \
    networkx pillow MDAnalysis ase
```

A tabela abaixo descreve o propósito de cada biblioteca:

Biblioteca	Finalidade
<code>numpy</code>	Arrays numéricos e álgebra linear
<code>scipy</code>	Algoritmos científicos (integração, otimização, estatística)
<code>matplotlib</code>	Geração de gráficos e visualizações
<code>pandas</code>	Manipulação de dados tabulares (DataFrames)
<code>seaborn</code>	Gráficos estatísticos de alta qualidade
<code>scikit-learn</code>	Aprendizado de máquina clássico
<code>jupyter</code>	Notebooks interativos (Jupyter Notebook / Lab)
<code>ipython</code>	Console Python interativo aprimorado
<code>sympy</code>	Matemática simbólica
<code>tqdm</code>	Barras de progresso em loops
<code>h5py</code>	Leitura/escrita de arquivos HDF5
<code>pyarrow</code>	Leitura/escrita de arquivos Parquet e Arrow
<code>statsmodels</code>	Modelos estatísticos e econométricos
<code>networkx</code>	Análise de grafos e redes
<code>pillow</code>	Processamento de imagens
<code>MDAnalysis</code>	Análise de trajetórias de dinâmica molecular
<code>ase</code>	Atomic Simulation Environment (materiais e moléculas)

Tabela 4: Bibliotecas Python instaladas no ambiente `ilumpy`.

6.4 Ativando o Ambiente Automaticamente

Para não precisar digitar `conda activate ilumpy` a cada login, adicione a ativação ao arquivo `~/.bashrc`:

```
$ nano ~/.bashrc
```

Adicione a linha abaixo ao final do arquivo:

```
conda activate ilumpy
```

Salve com `Ctrl+O` e saia com `Ctrl+X`. Na próxima vez que fizer login, o ambiente `ilumpy` já estará ativo.

6.5 Comandos Úteis do Conda

```
$ conda activate ilumpy      # ativa o ambiente
$ conda deactivate          # desativa (volta para base)
$ conda env list            # lista todos os ambientes
$ conda list                # lista pacotes do ambiente ativo
$ pip list                  # idem, para pacotes instalados via pip
```

```
$ pip install nome_pacote      # instala novo pacote
$ pip show numpy               # informacoes sobre um pacote
```

★ Dica

Sempre ative o ambiente correto **antes** de submeter um job que rode Python. No script `.sh`, inclua `conda activate ilumpy` logo após carregar os módulos necessários. Veja os exemplos na Seção 7.5.

7 Gerenciador de Filas Slurm

O **Slurm** é o software que gerencia e distribui os recursos do cluster entre os usuários. Você não executa seus programas diretamente; em vez disso, escreve um **script de job** que descreve os recursos necessários (CPUs, memória, tempo, GPUs) e o submete à fila. O Slurm decide quando e em qual nó o job irá rodar.

7.1 Filas (Partitions) Disponíveis

Fila	Default	Nós	CPUs Totais	Obs.
cpu	Sim	work0–3, work7	416	Apenas CPU
gpu	Não	work1, work4–6, work8	224	CPU + GPU

Tabela 5: Partições (filas) do Cluster Heisenberg.

△ Atenção

A fila `cpu` é a padrão. Se seu job precisar de GPU, especifique explicitamente `--partition=gpu` e `--gres=gpu:1` no script.

7.2 Comandos Essenciais do Slurm

Comando	Função
<code>sinfo</code>	Ver estado das filas e nós
<code>squeue</code>	Ver jobs na fila
<code>sbatch script.sh</code>	Submeter um job (modo batch)
<code>srun</code>	Executar interativamente
<code>scancel JOBID</code>	Cancelar um job
<code>sacct</code>	Ver histórico de jobs
<code>scontrol show job JOBID</code>	Detalhes de um job específico

Tabela 6: Comandos principais do Slurm.

7.2.1 Verificar estado das filas

```
$ sinfo
PARTITION  AVAIL  TIMELIMIT  NODES  STATE  NODELIST
cpu*       up     infinite   2      mix    work[0-1]
cpu*       up     infinite   2      alloc  work[2-3]
```

```
gpu      up      infinite  3      mix      work[1,4-5]
gpu      up      infinite  2      idle     work[6,8]
```

Os estados dos nós significam:

- `idle` — nó livre, pronto para receber jobs;
- `mix` — parcialmente ocupado (alguns CPUs livres);
- `alloc` — totalmente ocupado;
- `down` — nó fora de serviço.

7.2.2 Ver jobs em execução

```
$ squeue
JOBID   USER      NAME            PARTITION  STATE   TIME    NODES
9426    joao      simulacao.sh    cpu        RUNNING 40:13   1
9427    maria     analise.sh      gpu        PENDING 0:00    1

$ squeue -u meu_usuario # apenas seus jobs
```

7.3 Escrevendo um Script de Job

Um script de job é um arquivo de texto com extensão `.sh` contendo:

1. Diretivas Slurm (iniciadas com `#SBATCH`);
2. Comandos para carregar módulos;
3. O comando do seu programa.

7.3.1 Parâmetros Principais do #SBATCH

Parâmetro	Descrição
<code>--job-name=nome</code>	Nome do job (aparece no squeue)
<code>--partition=cpu</code>	Fila a usar (<code>cpu</code> ou <code>gpu</code>)
<code>--nodes=1</code>	Número de nós
<code>--ntasks=1</code>	Número de tarefas MPI
<code>--cpus-per-task=8</code>	CPUs por tarefa (threads OpenMP)
<code>--mem=16G</code>	Memória total por nó
<code>--time=12:00:00</code>	Tempo máximo (HH:MM:SS)
<code>--gres=gpu:1</code>	Solicitar 1 GPU
<code>--output=saida.out</code>	Arquivo de saída padrão
<code>--error=erros.err</code>	Arquivo de erros
<code>--mail-type=END,FAIL</code>	Notificação por e-mail
<code>--mail-user=email@ilum.br</code>	E-mail para notificações

Tabela 7: Principais diretivas `#SBATCH`.

7.4 Exemplos de Scripts de Job

7.4.1 Job Simples (CPU, serial)

```
1 #!/bin/bash
2 #SBATCH --job-name=meu_primeiro_job
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=1
```



```
5 #SBATCH --ntasks=1
6 #SBATCH --cpus-per-task=4
7 #SBATCH --mem=8G
8 #SBATCH --time=02:00:00
9 #SBATCH --output=saida_%j.out
10 #SBATCH --error=erros_%j.err
11
12 # Carregar modulos necessarios
13 module load intel/compiler/2025.3.0
14
15 # Executar o programa
16 echo "Job iniciado: $(date)"
17 ./meu_programa input.dat
18 echo "Job concluido: $(date)"
```

O %j no nome do arquivo será substituído automaticamente pelo número do job (JOBID), evitando sobrescrever saídas de execuções anteriores.

7.4.2 Job com MPI (múltiplos processos)

```
1 #!/bin/bash
2 #SBATCH --job-name=simulacao_mpi
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=2
5 #SBATCH --ntasks=16
6 #SBATCH --cpus-per-task=8
7 #SBATCH --mem=64G
8 #SBATCH --time=24:00:00
9 #SBATCH --output=mpi_%j.out
10 #SBATCH --error=mpi_%j.err
11
12 module load openmpi/5.0.9
13 module load intel/mkl/2025.3
14
15 srun ./programa_mpi input.dat
```

7.4.3 Job com GPU

```
1 #!/bin/bash
2 #SBATCH --job-name=treinamento_gpu
3 #SBATCH --partition=gpu
4 #SBATCH --nodes=1
5 #SBATCH --ntasks=1
6 #SBATCH --cpus-per-task=8
7 #SBATCH --gres=gpu:1
8 #SBATCH --mem=32G
9 #SBATCH --time=08:00:00
10 #SBATCH --output=gpu_%j.out
11 #SBATCH --error=gpu_%j.err
12
13 module load cuda/12.4
14
15 echo "GPU disponivel:"
16 nvidia-smi
17
18 python treinar_modelo.py --epochs 100
```

7.4.4 Job GROMACS (exemplo completo)

```
1 #!/bin/bash
2 #SBATCH --job-name=gromacs_md
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=1
5 #SBATCH --ntasks=1
6 #SBATCH --cpus-per-task=32
7 #SBATCH --mem=64G
8 #SBATCH --time=48:00:00
9 #SBATCH --output=gromacs_%j.out
10 #SBATCH --error=gromacs_%j.err
11 #SBATCH --mail-type=END,FAIL
12 #SBATCH --mail-user=meu@email.com.br
13
14 module load gromacs/2026.0
15
16 export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
17
18 # Etapa de equilibracao
19 gmx mdrun -v -deffnm equilibracao -ntmpi 1 \
20         -ntomp $OMP_NUM_THREADS
21
22 # Producao
23 gmx mdrun -v -deffnm producao -ntmpi 1 \
24         -ntomp $OMP_NUM_THREADS
```

7.4.5 Job VASP

```
1 #!/bin/bash
2 #SBATCH --job-name=vasp_dft
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=1
5 #SBATCH --ntasks=32
6 #SBATCH --mem=128G
7 #SBATCH --time=72:00:00
8 #SBATCH --output=vasp_%j.out
9 #SBATCH --error=vasp_%j.err
10
11 module load vasp/6.2.0
12
13 mpirun -np $SLURM_NTASKS vasp_std
```

7.5 Jobs Python

○ Informação

Python é serial por padrão. O interpretador CPython possui uma trava interna chamada **GIL** (*Global Interpreter Lock*) que impede a execução simultânea de múltiplos *threads* Python no mesmo processo. Isso significa que, na maior parte dos casos, um script Python utiliza **apenas 1 CPU** por vez, independentemente de quantas CPUs você reserve no Slurm. O comportamento muda quando o script usa bibliotecas que realizam processamento paralelo internamente, como:

- **NumPy / SciPy / scikit-learn** — operações vetorizadas podem usar múltiplos núcleos via BLAS/OpenBLAS em background;
- **multiprocessing / joblib / concurrent.futures** — paralelismo explícito usando múl-

tiplos processos;

- **PyTorch / TensorFlow** (com GPU) — treinamento em GPU requer a fila `gpu` e `--gres=gpu:1`.

Se o seu script não usa nenhuma dessas estratégias, solicite **apenas 1 CPU** e não desperdice recursos do cluster.

7.5.1 Job Python Serial

Use este modelo para scripts que **não** utilizam paralelismo explícito. A flag `-u` no comando Python desativa o buffer de saída, garantindo que as mensagens apareçam no arquivo de saída em tempo real (sem atraso).

```
1 #!/bin/bash
2 #SBATCH --job-name=python_serial
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=1
5 #SBATCH --ntasks=1
6 #SBATCH --cpus-per-task=1
7 #SBATCH --mem=8G
8 #SBATCH --time=04:00:00
9 #SBATCH --output=python_%j.out
10 #SBATCH --error=python_%j.err
11
12 # Ativa o ambiente Python com as bibliotecas instaladas
13 conda activate ilumpy
14
15 echo "Inicio: $(date)"
16 echo "No: $(hostname)"
17
18 # -u: saída sem buffer (aparece em tempo real no arquivo .out)
19 python -u script.py > script.out
20
21 echo "Fim: $(date)"
```

★ Dica

O redirecionamento `> script.out` grava a saída do `print` do Python em `script.out`, enquanto erros e exceções vão para o arquivo `.err` definido no `#SBATCH`. Assim você tem os resultados e os erros separados, facilitando a depuração.

7.5.2 Job Python Paralelo

Use este modelo quando o script utiliza múltiplos processos ou threads para aproveitar mais de um núcleo — por exemplo com `multiprocessing`, `joblib` ou operações pesadas de NumPy/SciPy que usam BLAS em paralelo.

```
1 #!/bin/bash
2 #SBATCH --job-name=python_paralelo
3 #SBATCH --partition=cpu
4 #SBATCH --nodes=1
5 #SBATCH --ntasks=1
6 #SBATCH --cpus-per-task=16
7 #SBATCH --mem=32G
8 #SBATCH --time=08:00:00
9 #SBATCH --output=python_%j.out
10 #SBATCH --error=python_%j.err
```

```

11
12 conda activate ilumpy
13
14 # Informa ao script quantas CPUs foram alocadas pelo Slurm
15 export N_CPUS=$SLURM_CPUS_PER_TASK
16
17 echo "Inicio: $(date)"
18 echo "No: $(hostname)"
19 echo "CPUs disponiveis: $N_CPUS"
20
21 python -u script.py > script.out
22
23 echo "Fim: $(date)"

```

No script Python, leia a variável de ambiente para saber quantos núcleos usar:

```

1 import os
2 from joblib import Parallel, delayed
3
4 n_cpus = int(os.environ.get("N_CPUS", 1))
5 print(f"Usando {n_cpus} CPUs")
6
7 # Exemplo com joblib: executa 'minha_funcao' em paralelo
8 resultados = Parallel(n_jobs=n_cpus)(
9     delayed(minha_funcao)(i) for i in range(100)
10 )

```

△ Atenção

Solicite apenas o número de CPUs que seu código **realmente** vai usar. Alocar 16 CPUs para um script serial desperdiça recursos e atrasa outros usuários na fila. Em caso de dúvida, comece com 1 CPU e aumente conforme necessário após verificar com [sacct](#).

7.6 Submetendo e Acompanhando o Job

```

# Submeter o job
$ sbatch meu_script.sh
Submitted batch job 9430

# Verificar se esta na fila
$ squeue -u meu_usuario
JOBID  USER  NAME                STATE    TIME
9430   joao  meu_primeiro_job    PENDING  0:00

# Acompanhar a saida em tempo real
$ tail -f saida_9430.out

# Cancelar o job
$ scancel 9430

```

7.7 Sessão Interativa

Para testar comandos, depurar scripts ou usar programas interativos, solicite uma sessão interativa em vez de submeter um job:

```

$ srun --partition=cpu --nodes=1 --ntasks=4 --mem=8G \
    --time=01:00:00 --pty bash

```

```
# Com GPU:
$ srun --partition=gpu --gres=gpu:1 --ntasks=4 \
    --mem=16G --time=02:00:00 --pty bash
```

△ Atenção

Sessões interativas também consomem recursos do cluster. Encerre-as com `exit` assim que terminar o trabalho.

7.8 Variáveis de Ambiente do Slurm

Dentro do script de job, o Slurm disponibiliza variáveis úteis:

Variável	Conteúdo
<code>\$SLURM_JOB_ID</code>	ID do job
<code>\$SLURM_JOB_NAME</code>	Nome do job
<code>\$SLURM_NTASKS</code>	Número total de tarefas
<code>\$SLURM_CPUS_PER_TASK</code>	CPUs por tarefa
<code>\$SLURM_MEM_PER_NODE</code>	Memória por nó (MB)
<code>\$SLURM_NODELIST</code>	Lista de nós alocados
<code>\$SLURM_SUBMIT_DIR</code>	Diretório de onde sbatch foi chamado

Tabela 8: Variáveis de ambiente do Slurm.

8 Boas Práticas

8.1 Organização de Arquivos

Mantenha sua área de trabalho organizada. Uma estrutura sugerida:

```
~/
|-- projetos/
|   |-- simulacao_proteina/
|   |   |-- input/
|   |   |-- output/
|   |   |-- scripts/
|   |   |-- logs/
|   |-- calculo_dft/
|   |-- ...
|-- scripts uteis/
```

8.2 Estimativa de Recursos

- Comece com jobs pequenos para calibrar o tempo de execução.
- Não solicite mais memória ou CPUs do que seu programa realmente usa.
- Monitore o uso real com `sstat` ou `sacct` após o job terminar.

```
$ sacct -j 9430 --format=JobID,Elapsed,CPUTime,MaxRSS,State
```

8.3 Cotas e Limites

Cada estudante pode utilizar até **16 CPUs simultâneas** no cluster. Casos excepcionais que demandem mais recursos podem ser avaliados pela equipe de TI mediante abertura de chamado em <https://web-ilum.cnpem.br/chamado>.

8.4 Etiqueta no Uso do Cluster

1. **Nunca** execute jobs pesados no nó de login.
2. Estime o tempo necessário e coloque `--time` realista (nem muito apertado, nem exagerado).
3. Cancele jobs com `scancel` se perceber que não precisará mais deles.
4. Remova arquivos temporários e de saída desnecessários regularmente.
5. Em caso de dúvidas técnicas ou problemas, abra um chamado em <https://web-ilum.cnpem.br/chamado>.

9 Guia Rápido de Referência

9.1 Cheatsheet de Linux

Comando	O que faz
<code>pwd</code>	Mostra o diretório atual
<code>ls -lh</code>	Lista arquivos com detalhes
<code>cd pasta/</code>	Entra na pasta
<code>cd ..</code>	Volta um nível
<code>mkdir nome</code>	Cria diretório
<code>cp a b</code>	Copia arquivo a para b
<code>mv a b</code>	Move/renomeia a para b
<code>rm arquivo</code>	Remove arquivo
<code>nano arquivo</code>	Edita arquivo
<code>cat arquivo</code>	Exibe arquivo
<code>less arquivo</code>	Exibe com paginação
<code>grep "texto"arq</code>	Busca texto no arquivo
<code>scp arq user@host:dir/</code>	Envia arquivo via SSH
<code>Ctrl+C</code>	Cancela comando atual
<code>Tab</code>	Autocompleta
<code>man comando</code>	Manual do comando

Tabela 9: Referência rápida de comandos Linux.

9.2 Cheatsheet do Slurm

Comando	O que faz
<code>sinfo</code>	Estado das filas e nós
<code>squeue</code>	Jobs na fila
<code>squeue -u usuario</code>	Seus jobs
<code>sbatch script.sh</code>	Submeter job
<code>scancel JOBID</code>	Cancelar job

Comando	O que faz
<code>scontrol show job JOBID</code>	Detalhes do job
<code>sacct -j JOBID</code>	Histórico e uso de recursos
<code>srun --pty bash</code>	Sessão interativa

Tabela 10: Referência rápida de comandos Slurm.

9.3 Cheatsheet de Módulos

Comando	O que faz
<code>module avail</code>	Lista módulos disponíveis
<code>module load nome/versao</code>	Carrega módulo
<code>module list</code>	Módulos carregados
<code>module unload nome</code>	Descarrega módulo
<code>module purge</code>	Descarrega todos
<code>module show nome</code>	Mostra o que o módulo altera

Tabela 11: Referência rápida de Environment Modules.

9.4 Cheatsheet do Conda / Python

Comando	O que faz
<code>conda activate ilumpy</code>	Ativa o ambiente ilumpy
<code>conda deactivate</code>	Desativa o ambiente atual
<code>conda env list</code>	Lista todos os ambientes
<code>conda list</code>	Lista pacotes do ambiente ativo
<code>pip install pacote</code>	Instala pacote no ambiente ativo
<code>pip list</code>	Lista pacotes instalados via pip
<code>pip show pacote</code>	Informações sobre um pacote
<code>python -u script.py</code>	Executa Python sem buffer de saída
<code>python -u script.py > out</code>	Redireciona saída para arquivo

Tabela 12: Referência rápida de Conda e Python.

10 Suporte e Contato

Onde buscar ajuda

Sistema de chamados: <https://web-ilum.cnpem.br/chamado>

Use para relatar problemas técnicos, solicitar instalação de software, pedir aumento de cota ou qualquer dificuldade com o cluster.

Reset de senha: <http://172.20.60.13>

Acesse dentro da rede Ilum ou via VPN para redefinir sua senha.

Endereço do cluster: heisenberg.cnpem.br

Ao abrir um chamado, inclua sempre:

- Seu nome de usuário;
- O JOBID do job com problema (se aplicável);
- A mensagem de erro completa;
- O conteúdo do arquivo `.err` gerado pelo job.

A Glossário

Cluster

Conjunto de computadores interconectados que trabalham como uma única máquina.

CPU (*Central Processing Unit*)

Processador central; realiza cálculos de propósito geral.

Ambiente Virtual (*conda/venv*)

Espaço isolado com versão própria do Python e bibliotecas, sem interferir no sistema ou em outros projetos.

conda

Gerenciador de pacotes e ambientes virtuais; base do Miniconda e Anaconda.

GIL (*Global Interpreter Lock*)

Trava do CPython que impede execução simultânea de múltiplos *threads* Python; torna scripts comuns essencialmente seriais.

GPU (*Graphics Processing Unit*)

Processador gráfico; realiza cálculos paralelos massivos (ML, simulações GPU).

HPC (*High-Performance Computing*)

Computação de alto desempenho.

Miniconda

Distribuição minimalista do conda; permite instalar o gerenciador de pacotes Python sem uma distribuição completa como o Anaconda.

Job Tarefa submetida ao Slurm para execução nos nós de trabalho.

MPI (*Message Passing Interface*)

Padrão para comunicação entre processos em múltiplos nós.

Módulo (*Environment Module*)

Mecanismo para carregar/descarregar softwares e suas dependências dinamicamente.

Nó (*Node*)

Cada computador individual dentro do cluster.

Nó de login (*login node*)

Ponto de acesso ao cluster; não deve ser usado para cálculos pesados.

OpenMP

API para paralelismo de memória compartilhada (multithreading).

Partição (*Partition*)

Fila de jobs com regras específicas de acesso e recursos.

QOS (*Quality of Service*)

Política que limita o uso de recursos por usuário ou grupo.

SSH (*Secure Shell*)

Protocolo criptografado para acesso remoto a servidores.

Slurm

Sistema de gerenciamento de recursos e filas de jobs usado no Heisenberg.

B Checklist do Primeiro Uso

- ☐ Obtive minha conta na Ilum/CNPEM.
- ☐ Abri o PowerShell (Windows) ou o terminal (Linux/macOS).
- ☐ Me conectei ao cluster: `ssh usuario@heisenberg.cnpem.br`.
- ☐ Explorei minha pasta home com `ls` e `pwd`.
- ☐ Listei os módulos com `module avail`.
- ☐ Instalei o Miniconda na minha home.
- ☐ Criei o ambiente `ilumpy` com Python 3.11.
- ☐ Instalei as bibliotecas científicas com `pip install`.
- ☐ Verifiquei as filas com `sinfo`.
- ☐ Escrevi e submeti meu primeiro job com `sbatch`.
- ☐ Acompanhei o job com `squeue` e `tail -f saida.out`.